جزوه جامع و بی‌نظیر فصل دوم (بخش دوم: رابط‌ها و فراخوانی‌های سیستمی)

این جزوه بر اساس بخش دوم از فصل دوم فایل ارسالی شما و با هدف پوشش **کامل، عمیق و مفهومی** تمام مطالب، تعاریف فنی و مثال‌های ذکر شده تهیه شده است. هدف، درک کامل مفاهیم برای کسب بالاترین نمره است.

۲-۲: رابط کاربر و سیستم‌عامل (User and Operating-System Interface)

پس از آشنایی با سرویس‌های سیستم‌عامل، اکنون به بررسی **چگونگی دسترسی** به این سرویس‌ها می‌پردازیم. کاربران و برنامه‌ها از طریق رابط‌های مشخصی با سیستم‌عامل تعامل می‌کنند.

۲-۲-۱: مفسرهای فرمان (Command Interpreters)

این رابط که به آن **رابط خط فرمان (CLI)** یا **پوسته (Shell)** نیز گفته می‌شود، یکی از قدیمی‌ترین و قدرتمندترین روش‌های تعامل با سیستم‌عامل است.

- **عملکرد:** کاربر دستورات را به صورت متنی تایپ می‌کند و مفسر فرمان آن‌ها را دریافت، تفسیر و اجرا می‌کند.
- **پوسته (Shell):** در سیستم‌عامل‌هایی مانند یونیکس و لینوکس، مفسر فرمان یک برنامه مجزاست که می‌توان آن را تعویض کرد. به همین دلیل به آن "پوسته" می‌گویند، زیرا هسته سیستم‌عامل را در بر می‌گیرد. پوسته‌های معروفی مانند C Shell (`csh`)، Bourne-Again Shell (`bash`) و Korn Shell (`ksh`) وجود دارند که انتخاب بین آن‌ها اغلب به سلیقه کاربر بستگی دارد.
- **دو روش پیاده‌سازی دستورات:**

1. **کد دستورات داخل مفسر:** در این روش، کد مربوط به هر دستور (مانند `delete`) بخشی از خود برنامه مفسر فرمان است. این روش باعث بزرگ شدن حجم مفسر می‌شود و افزودن دستورات جدید به آن دشوار است.
2. **دستورات به عنوان برنامه‌های مجزا:** این روش که در یونیکس و سیستم‌عامل‌های مشابه استفاده می‌شود، بسیار انعطاف‌پذیرتر است. در اینجا، مفسر فرمان (پوسته) هیچ کدی برای اجرای دستورات ندارد. هر دستور (مانند `rm`, `ls`, `cp`) یک برنامه اجرایی جداگانه روی دیسک است.
   - **فرآیند اجرا:** وقتی کاربر دستوری مانند `rm file.txt` را تایپ می‌کند، پوسته:
     1. فایلی به نام `rm` را در مسیرهای جستجوی سیستم پیدا می‌کند.
     2. یک پردازه جدید ایجاد می‌کند.
     3. برنامه `rm` را در حافظه بارگذاری و اجرا می‌کند.
     4. `file.txt` را به عنوان یک پارامتر به برنامه `rm` ارسال می‌کند.
   - **مزیت:** این روش به برنامه‌نویسان اجازه می‌دهد تا به راحتی با نوشتن برنامه‌های جدید، دستورات جدیدی به سیستم اضافه کنند، بدون اینکه نیازی به تغییر خود پوسته باشد.

۲-۲-۲: رابط کاربر گرافیکی (Graphical User Interfaces - GUI)

GUI با استفاده از استعاره **میزکار (Desktop)**، کار با کامپیوتر را برای کاربران عادی بسیار ساده و بصری کرده است.

- **عناصر اصلی:**
  - **سیستم پنجره‌ای (Windowing System):** برای نمایش چندین برنامه به صورت همزمان.

- **اشاره‌گر (Pointer):** که با ماوس کنترل می‌شود.
- **آیکون‌ها (Icons):** تصاویری که نماینده فایل‌ها, پوشه‌ها (Folders) یا برنامه‌ها هستند.
- **منوها (Menus):** لیستی از دستورات و گزینه‌ها.

- **تاریخچه:** GUI برای اولین بار در مرکز تحقیقاتی **Xerox PARC** در دهه ۱۹۷۰ ابداع شد و با کامپیوتر **Apple Macintosh** در دهه ۱۹۸۰ به محبوبیت جهانی رسید. ویندوز مایکروسافت نیز در ابتدا یک رابط گرافیکی بود که روی سیستم‌عامل متنی MS-DOS اجرا می‌شد.
- **در دنیای یونیکس/لینوکس:** محیط‌های میزکار قدرتمندی مانند **KDE** و **GNOME** رابط‌های گرافیکی کاملی را برای این سیستم‌عامل‌ها فراهم کرده‌اند.

۲-۲-۳ : رابط صفحه نمایش لمسی *(Touch-Screen Interface)*

این رابط، روش اصلی تعامل در دستگاه‌های موبایل مدرن است.

- **عملکرد:** کاربران با **ژست‌های حرکتی (Gestures)** مانند لمس کردن، کشیدن و زوم کردن با انگشتان، با سیستم‌عامل ارتباط برقرار می‌کنند.
- **صفحه کلید مجازی:** اکثر دستگاه‌های لمسی، صفحه کلید را به صورت نرم‌افزاری روی خود نمایشگر شبیه‌سازی می‌کنند.

۲-۲-۴ : انتخاب رابط مناسب

- **CLI:** مورد علاقه **مدیران سیستم (System Administrators)** و **کاربران حرفه‌ای (Power Users)** است، زیرا:
  - **سریع و کارآمد:** دسترسی مستقیم و سریع به تمام قابلیت‌های سیستم.
  - **قابلیت اسکریپت‌نویسی:** می‌توان دنباله‌ای از دستورات را در یک فایل به نام **اسکریپت پوسته (Shell Script)** ذخیره کرد و آن را مانند یک برنامه اجرا نمود. این ویژگی برای خودکارسازی کارهای تکراری بسیار قدرتمند است.
- **GUI:** مورد علاقه اکثر کاربران عادی است، زیرا:
  - **یادگیری آسان و بصری:** نیاز به حفظ کردن دستورات ندارد.
- **رویکرد ترکیبی:** سیستم‌عامل‌های مدرن مانند **macOS** و توزیع‌های مختلف لینوکس، هر دو رابط CLI و GUI را به صورت یکپارچه ارائه می‌دهند تا کاربران بسته به نیاز خود بتوانند از هر دو استفاده کنند.

۲-۳ : فراخوانی‌های سیستمی (System Calls)

فراخوانی‌های سیستمی، مهم‌ترین مفهوم در این فصل هستند. آن‌ها **درگاه ورودی** برنامه‌های کاربر به دنیای قدرتمند **هسته (Kernel)** سیستم‌عامل هستند.

- **تعریف:** فراخوانی سیستمی، یک **رابط برنامه‌نویسی (API)** است که سرویس‌های ارائه شده توسط سیستم‌عامل را در اختیار برنامه‌های در حال اجرا قرار می‌دهد.
- **چرا به فراخوانی سیستمی نیاز داریم؟** همانطور که در فصل قبل دیدیم، برنامه‌های کاربر در **مد کاربر (User Mode)** با دسترسی محدود اجرا می‌شوند. بسیاری از عملیات حیاتی (مانند کار با سخت‌افزار I/O یا مدیریت حافظه) نیازمند اجرای **دستورالعمل‌های ممتاز (Privileged Instructions)** هستند که فقط در **مد هسته (Kernel Mode)** قابل اجرا هستند. فراخوانی سیستمی، تنها راه **قانونمند** و **کنترل‌شده** برای یک برنامه است تا از سیستم‌عامل بخواهد یک عملیات ممتاز را به نمایندگی از آن انجام دهد.

- **زبان پیاده‌سازی:** فراخوانی‌های سیستمی معمولاً به صورت توابعی در زبان‌های برنامه‌نویسی سطح بالا مانند **C** یا **C++** در دسترس برنامه‌نویسان قرار می‌گیرند، هرچند که پیاده‌سازی داخلی آن‌ها در هسته ممکن است شامل کدهای سطح پایین به زبان **اسمبلی** باشد.

۱-۳-۲: یک مثال کاربردی: کپی کردن یک فایل

بیایید فرآیند ساده کپی کردن محتوای یک فایل به فایل دیگر را از دیدگاه فراخوانی‌های سیستمی بررسی کنیم. برای این کار، برنامه ما به ترتیب زیر به سرویس‌های سیستم‌عامل نیاز دارد:

1. **دریافت نام فایل‌ها:** برنامه باید نام فایل ورودی و خروجی را از کاربر (مثلاً از خط فرمان) دریافت کند. این کار نیازمند فراخوانی‌های سیستمی برای خواندن از دستگاه ورودی است.
2. **باز کردن فایل ورودی:** برنامه باید فایل ورودی را برای خواندن باز کند. این کار با یک فراخوانی سیستمی (`open`) انجام می‌شود. سیستم‌عامل بررسی می‌کند که آیا فایل وجود دارد و آیا برنامه مجوز خواندن آن را دارد یا خیر.
3. **ایجاد فایل خروجی:** اگر فایل خروجی وجود نداشته باشد، باید ایجاد شود. این کار نیز با یک فراخوانی سیستمی (`create` یا `open` با پارامترهای خاص) انجام می‌شود.
4. **حلقه خواندن و نوشتن:** برنامه در یک حلقه، به طور مکرر:
    - با یک فراخوانی سیستمی (`read`)، یک بلوک از داده را از فایل ورودی می‌خواند.
    - با یک فراخوانی سیستمی (`write`)، همان بلوک داده را در فایل خروجی می‌نویسد. این حلقه تا زمانی که تمام محتوای فایل ورودی خوانده شود، ادامه می‌یابد.
1. **بستن فایل‌ها:** پس از اتمام کپی، برنامه باید با فراخوانی‌های سیستمی (`close